

OFFICIAL TECHNICAL GUIDE & SYSTEM BLUEPRINT

SLA SkillUp - NMA Aptitude Portal

Comprehensive End-to-End System Manual: Architecture, Database Schemas, API Endpoints, Frontend Workflows, Business Rules, and Operational Reference.

SYSTEM SPECIFICATIONS

- **Application Name:** SLA SkillUp Portal (NMA Aptitude Portal)
- **Architecture:** Mobile App (React Native/Expo) + Admin Web (React/Vite) + Backend API (Node/Express)
- **Database:** MongoDB Atlas with Mongoose ODM 8
- **Mobile Distribution:** Standalone Android APK (Expo EAS Build)
- **Date of Publication:** September 2026

Crafted for SLA Softlogic Systems / NMA Faculty, Instructors & Engineering Team

1. Executive Summary & Project Purpose

SLA SkillUp Portal is an end-to-end aptitude learning, weekly assessment, and corporate recruitment training platform designed for **Softlogic Academy (SLA)** and **NMA** students. The portal provides a seamless bridge between academic training and top multinational corporation (MNC) hiring assessments (TCS, Infosys, Wipro, Cognizant, Zoho, Accenture, Amazon).

Core Target User Roles:

- **Students (Android Mobile App):** Register with email/phone, attend weekly scheduled exams with live countdown timers, practice 100+ topic drills, take 10-model mock assessments, view scores, review mistakes, and engage with gamification (streaks, coins, spin wheel, avatar frames).
- **Admin & Faculty (Web Dashboard):** Manage comprehensive question banks, upload question files (Excel, Word, CSV), schedule weekly tests, grant/revoke mock access, auto-distribute exam models, monitor results, and manage student accounts.

Key Business Objectives Accomplished:

1. **Anti-Cheating Integrity:** 24-hour review lock for official weekly tests stops students from sharing question solutions during active test windows.
2. **Instant Self-Practice:** Topic Practice mode delivers immediate answer explanations and mistake reviews with zero delay.
3. **Universal File Uploads:** Automatic parsing of Word (.docx), Excel (.xlsx), and CSV files directly into structured questions with options and explanations.
4. **Emergency Batch Undo & Delete:** Total deletion by week or topic, plus "Undo Latest Upload" to instantly revert accidental uploads without affecting older questions.
5. **Dual-Channel Password Recovery:** Real Email OTP (Nodemailer HTML mailer) and Mobile SMS OTP.
6. **Student Profile & Account Management:** Full admin control to inspect student test records, edit details, and reset passwords directly.

2. System Architecture & Tech Stack

The system is built on a modern, decoupled 3-tier client-server architecture:

Tier / Module	Technologies	Role & Key Responsibilities
Mobile App (Frontend)	React Native Expo Router AsyncStorage Expo SDK 57 Axios	Primary client for students. Features exam countdown timers, test submitters, separate history tabs, instant mistake reviews, gamification engine, and offline state protection.
Admin Portal (Frontend)	React 18 React Router 6 Lucide Icons Vite 5	Responsive web management dashboard for trainers and administrators. Manages weekly test schedules, bulk question uploads, delete hubs, student profiles, and mock model distribution.
REST API Server (Backend)	Node.js BcryptJS Nodemailer Express.js JWT Multer	Scalable backend hosting JWT auth middleware, universal file parser (docx/xlsx/csv), 24h review lock evaluation, grading algorithms, and transactional MongoDB operations.
Database Tier	MongoDB Atlas Mongoose ODM 8	NoSQL document store containing collections for students, questions, weekly tests, test submissions, mock questions, and gamification data.
Cloud Build Pipeline	Expo EAS Build	Automated cloud build infrastructure compiling the React Native application into standalone Android APKs (.apk) ready for direct phone installation.

3. Codebase Directory Map

The repository root is organized into three main decoupled workspaces:

```

NMA1/
├── backend/                                # REST API Server (Node.js + Express)
│   ├── src/
│   │   ├── controllers/                   # Route controllers (auth, admin, exam, weeklyTest, mockTest)
│   │   ├── models/                       # Mongoose ODM schemas (Student, Question, Result, etc.)
│   │   ├── routes/                       # Express router files
│   │   ├── middleware/                   # JWT auth guards (protect, adminOnly) & Multer upload
│   │   ├── services/                     # File parser service (universal parser for docx/xlsx/csv)
│   │   ├── utils/                        # Utilities (sendEmail.js HTML mailer)
│   │   ├── validators/                   # Input validation schemas
│   │   └── server.js                     # HTTP server entry point & DB bootstrap
│   ├── package.json
│   └── .env                               # Port, Mongo URI, JWT secret, SMTP configuration
├── admin-web/                             # Admin Web Management Console (React 18 + Vite)
│   ├── src/
│   │   ├── components/                   # AdminLayout, Navigation Sidebar, Top Header
│   │   ├── context/                     # AuthContext (admin login state & token storage)
│   │   ├── pages/                       # Admin screens:
│   │   │   ├── Dashboard.jsx             # KPI cards & system overview
│   │   │   ├── ManageQuestions.jsx       # Delete Hub (Weekly tests & Topic deletion)
│   │   │   ├── QuestionUpload.jsx        # Multi-file question upload & manual form
│   │   │   ├── WeeklyTests.jsx           # Weekly test schedule manager
│   │   │   ├── ThisWeekQuestions.jsx     # Live week question inspector
│   │   │   ├── StudentManagement.jsx     # Student Details, Edit Profile, Reset Pwd
│   │   │   ├── TestResults.jsx           # Weekly test submissions & manual marking
│   │   │   ├── MockTestSettings.jsx      # Mock test model question upload
│   │   │   └── MockTestResults.jsx       # Mock test student scores
│   │   ├── services/                    # Axios API client with bearer token interceptor
│   │   ├── App.jsx                      # Client-side router configuration
│   │   └── main.jsx                     # Vite React entry point
│   ├── package.json
│   └── vite.config.js
└── frontend/                             # Student Mobile App (React Native / Expo SDK 57)
    ├── app/                             # Expo Router directory & screen entry points
    └── src/
        ├── screens/                     # Mobile screens:
        │   ├── StudentLoginScreen.jsx
        │   ├── StudentRegisterScreen.jsx
        │   ├── StudentForgotPasswordScreen.jsx # Email OTP & Phone OTP
        │   ├── HomeScreen.jsx            # Dashboard, streaks, game coins
        │   ├── CategorySelectScreen.jsx   # Practice Topics + Practice History
        │   ├── TopicHistoryScreen.jsx     # Standalone topic practice history
        │   ├── ExamScreen.jsx            # Live test runner with timer
        │   ├── ResultScreen.jsx           # Score report & mistake review
        │   ├── PreviousTestsScreen.jsx    # Weekly test submissions list
        │   ├── MockTestDashboardScreen.jsx # Model 1-10 mock assessment portal
        │   └── MockExamScreen.jsx         # Corporate mock exam runner
    
```

```
| | └─ GameZoneScreen.jsx          # Spin wheel, streak claim, battles
| └─ services/                    # Mobile API services (authService, examService, api.js)
| └─ context/                     # AuthContext (student JWT session)
|   └─ components/                # Reusable mobile UI widgets
└─ app.json                       # Expo application configuration (name, package, icons)
└─ eas.json                       # EAS Build profiles (preview -> standalone APK)
└─ package.json
```

4. Database Architecture & Data Models

1. Student Model (`backend/src/models/Student.js`)

Field	Type	Description
<code>name</code>	String (Req)	Full legal name of the student
<code>email</code>	String (Req, Unique)	Lowercase email address used for login and OTP delivery
<code>password</code>	String (Req)	Encrypted hash using BcryptJS (10 salt rounds)
<code>phone</code>	String	Mobile phone number for SMS OTP recovery
<code>courseName</code>	String	Enrolled batch or course (e.g. Java Full Stack, Aptitude)
<code>trainerName</code>	String	Faculty trainer assigned to student
<code>status</code>	ACTIVE / INACTIVE	Active accounts can login; Inactive accounts are blocked
<code>mockTestAllowed</code>	Boolean	Access permission to write 10-model corporate mock exams
<code>assignedMockModel</code>	String	Assigned model ('Model 1' through 'Model 10')
<code>dailyStreak</code>	Number	Consecutive daily login streak counter
<code>gameCoins</code>	Number	In-app reward currency earned from exams and wheel spins
<code>inventory</code>	[String]	Unlocked cosmetic avatar profile frames

2. Question Model (`backend/src/models/Question.js`)

Field	Type	Description
<code>questionText</code>	String (Req)	The complete question problem statement
<code>options</code>	[String] (Size 4)	Multiple choice options: [Option A, Option B, Option C, Option D]
<code>correctAnswer</code>	String (Req)	The correct option text or matching choice
<code>explanation</code>	String	Comprehensive mathematical or logical explanation for review
<code>category</code>	String	Subject category: Quantitative, Logical, Verbal, Core
<code>topic</code>	String	Specific subtopic (e.g. Percentage, Time and Work, Syllogism)
<code>difficulty</code>	Easy/Med/Hard	Difficulty level classifier

Field	Type	Description
<code>weeklyTestId</code>	ObjectId (Ref)	Nullable. Set if question belongs to a specific weekly test
<code>batchId</code>	String	Timestamp identifier grouped per upload batch for batch undo

3. WeeklyTest Model (`backend/src/models/WeeklyTest.js`)

Field	Type	Description
<code>weekNumber</code>	Number (Req)	Sequential index (Week 1, Week 2, Week 3, etc.)
<code>weekName</code>	String	Descriptive tag (e.g. "Week 4 - Quantitative Drill")
<code>title</code>	String (Req)	Official examination title displayed to students
<code>totalQuestions</code>	Number	Dynamically tracked count of linked questions
<code>durationMinutes</code>	Number	Exam countdown duration in minutes
<code>passPercentage</code>	Number	Passing threshold (default: 50%)
<code>status</code>	ACTIVE/INACTIVE	Active tests appear on mobile dashboard; Inactive tests are hidden
<code>scheduledDate</code>	Date	Scheduled launch date of the exam

4. Result Model (`backend/src/models/Result.js`)

Field	Type	Description
<code>student</code>	ObjectId (Ref)	Foreign key pointing to the submitting student
<code>weeklyTestId</code>	ObjectId (Ref)	Nullable. Differentiates weekly tests from topic tests
<code>category</code>	String	Topic or test category
<code>score</code>	Number	Total marks scored
<code>total</code>	Number	Total questions in the exam
<code>correctAnswers</code>	Number	Total questions answered correctly
<code>wrongAnswers</code>	Number	Total questions answered incorrectly
<code>isPassed</code>	Boolean	Calculated automatically against pass percentage
<code>submittedAt</code>	Date	Timestamp used for 24-hour review unlock calculation
<code>answersJson</code>	String (JSON)	Serialized map of question IDs to student selected choices

5. Core Workflows & Business Rules

1. The 24-Hour Review Lock Engine

Security Purpose: During weekend examination windows, early students could leak correct answer keys to classmates. The 24-hour lock protects academic integrity.

- **Weekly Official Tests** (`weeklyTestId != null`): When a student submits, they immediately receive their score (e.g. 18/25) and pass/fail status. However, answer explanations and mistake reviews are locked for 24 hours from the exact moment of submission:

```
const now = Date.now();
const submissionTime = new Date(result.submittedAt).getTime();
const msSinceSubmission = now - submissionTime;
const TWENTY_FOUR_HOURS_MS = 24 * 60 * 60 * 1000;

const isStudent = req.user && req.user.role === 'student';
const isWeeklyTest = !!result.weeklyTestId;
const canViewMistakes = !isStudent || !isWeeklyTest || (msSinceSubmission >= TWENTY_FOUR_HOURS_MS)
```

If a student requests review before 24 hours expire, the API returns the exact remaining unlock duration in milliseconds and hides answer keys.

- **Topic Practice** (`weeklyTestId == null`): Topic Practice is for self-study and learning. Immediate mistake feedback is vital for learning. Therefore, **Topic Practice has NO lock** (0 seconds)—students review explanations instantly upon submission!

2. Separate History Portals

- **Previous Official Tests:** Accessed via `PreviousTestsScreen.jsx`. Queries backend for `{ weeklyTestId: { $ne: null } }`.
- **Topic Practice History:** Integrated directly into `CategorySelectScreen.jsx` with a dual-tab switcher:
[📚 Practice Topics] | [📄 Practice History]
Shows total practice sessions, scores, accuracy percentage, and an instant "Review Mistakes" button.

3. Real Email OTP & Mobile OTP Forgot Password

Nodemailer SMTP Integration: Direct delivery of 6-digit verification codes to the student's email inbox with branded HTML formatting.

1. Student selects  **Via Email Address** or  **Via Mobile Number**.
2. Backend generates a secure 6-digit OTP stored in-memory with a 10-minute expiry timestamp.

3. `sendEmail.js` dispatches the email via configured SMTP (e.g. Gmail App Password).
4. Student verifies the OTP and enters a new password.
5. Backend securely hashes the new password using Bcrypt (10 salt rounds) and updates MongoDB.

4. Question Delete Hub & Batch Undo

- **Weekly Test Total Delete:** Admin can delete all questions belonging to any specific week with one click. Automatically resets the test's `totalQuestions: 0`.
- **Topic Practice Total Delete:** Wipes all questions under a specific topic clean.
- **Undo Latest Upload (Accidental Upload Safety):** Deletes only questions uploaded in the most recent batch (matching latest `batchId`). Previously uploaded questions remain 100% safe.

5. Admin Student Details & Management

- **View Full Profile:** Full name, email, phone, course, trainer, join date, daily streak, total tests taken, passed count, average accuracy, and full history table.
- **Edit Profile:** Admin can update student name, email, phone, course, trainer, status (Active/Inactive), mock model, and directly reset the student's password.
- **Delete Account:** Allows deleting test or spam accounts.

6. Core API Endpoint Reference

Base Server URL: `http://localhost:5000/api`

HTTP	Endpoint	Role	Description
POST	<code>/auth/student/register</code>	Public	Student account registration
POST	<code>/auth/student/login</code>	Public	Student login, issues JWT session token
POST	<code>/auth/admin/login</code>	Public	Admin login, issues JWT session token
POST	<code>/auth/forgot-password/request</code>	Public	Sends 6-digit OTP to student email or phone
POST	<code>/auth/forgot-password/reset</code>	Public	Verifies OTP and resets password
GET	<code>/weekly-tests/active</code>	Student	Fetch all active scheduled weekly tests
GET	<code>/weekly-tests/student-history</code>	Student	Fetch student's official weekly test results
GET	<code>/exam/topic-history</code>	Student	Fetch student's practice topic test results
POST	<code>/exam/submit</code>	Student	Submit test answers & calculate marks
GET	<code>/admin/results/:id/details</code>	Both	Get test review (enforces 24h lock on weekly tests)
GET	<code>/admin/students/management</code>	Admin	Fetch all students with mock access info
PUT	<code>/admin/students/:id</code>	Admin	Update student profile & reset password
DELETE	<code>/admin/students/:id</code>	Admin	Delete student account
POST	<code>/admin/questions/upload-file</code>	Admin	Parse and upload questions from Excel, Word, CSV
POST	<code>/admin/questions/delete-by-scope</code>	Admin	Delete by week/topic or undo latest upload batch

7. How to Run, Test & Deploy

1. Running the Backend Server

```
cd backend
npm install
npm run dev    # Runs on http://localhost:5000 with nodemon live reload
```

2. Running the Admin Web Console

```
cd admin-web
npm install
npm run dev    # Runs on http://localhost:3000
npm run build  # Compiles production distribution to dist/ folder
```

3. Running the Student Mobile App

```
cd frontend
npm install
npx expo start # Launches Expo DevTools QR Code for Expo Go / Emulator
```

4. Compiling Standalone Android APK (EAS Build)

Builds the production Android APK (.apk) directly in Expo cloud:

```
cd frontend
npx eas-cli build -p android --profile preview --non-interactive
```

5. Backend Environment Configuration (backend/.env)

```
PORT=5000
MONGODB_URI=mongodb+srv://<user>:<password>@cluster0.mongodb.net/sla-skillup?retryWrites=true&w=majority
JWT_SECRET=sla_super_secret_jwt_key_2026

# Nodemailer SMTP Configuration
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=your_email@gmail.com
EMAIL_PASS=your_16_character_app_password
```

```
EMAIL_FROM="SLA SkillUp Portal <your_email@gmail.com>"
```



8. Production Artifacts & Quick Links

Current Production Build & Artifacts:

- **Latest Android Mobile APK Download:**

<https://expo.dev/artifacts/eas/qVJGInB-MzxVdnVF4G32zT87BIhPU9ZOrXDKpSNd6mA.apk>

- **Expo EAS Build ID:** `a6cad9ad-f219-46de-bb89-f89a5d9e67a7`

- **GitHub Remote:** <https://github.com/nagaarjun93/sla-skillup-portal.git>

- **Git Branch:** `main` (Commit: `dea3959`)

Summary of AI Engineering Work Delivered:

1. **Full Architecture Implementation:** Complete 3-tier system built, integrated, and verified.
2. **Integrity by Design:** 24-hour review lock for official weekly tests, combined with free instant-feedback topic drills.
3. **Disaster Recovery for Admin:** Universal multi-file parsing with instant single-click batch undo capabilities.
4. **Reliable User Experience:** Branded email OTP delivery, password resets, and robust student account management.
5. **Continuous Deployment:** Standalone Android APK ready for student installation.